AN AI POWERED SOFTWARE-BASED FIRE DETECTION SYSTEM FOR CCTV PLATFORMS.

FiremeX - Test Environment Setup

Project	FiremeX
Document	Document of environmental isolation for testing
Prepared By	Jayashini Jayaweera

1. Purpose

This is describes the QA-FiremeX repository's local QA services, connect them to a separately running FiremeX application, and execute the repository's API and UI automation.
The repository is primarily a QA/test repository. Its current Docker Compose configuration provisions PostgreSQL and Home Assistant; it does not provision the FiremeX backend, frontend, AI inference service, CCTV/NVR hardware, message queue, notification gateway, or mock fire-panel API.
The project architecture describes a CCTV/RTSP stream flowing through frame capture and preprocessing to PyTorch/YOLO inference, with detections and saved frames feeding the FiremeX backend, database, dashboard, and test integrations.
This QA repository does not include or deploy that complete runtime pipeline.

Environment Challenges

- Production Discrepancies: Test environments lack production-scale traffic, concealing concurrency bugs.
- Configuration Drift: Server settings altered over time lead to sudden unexpected failures.
- Device/Browser Fragmentation: Limited hardware availability causes teams to miss device-specific defects.

2. Environment profile

Use this environment for functional, integration, and regression testing with non-production data. Not connect it to live cameras, production databases, real emergency services, or customer notification accounts.

2.1 Supported local test workstation

- Windows 11 or a compatible development workstation.
- Docker Desktop with Docker Compose v2.
- Python 3 and pip for API/UI automation.
- A Chromium-based browser for Selenium UI tests.
- Network access to the separately deployed FiremeX frontend and backend.
- Additional disk and memory appropriate to the PostgreSQL data volume, Home Assistant, browser automation, and any separately hosted model service.

2.2 QA service inventory

Component	Intended local endpoint	Notes
PostgreSQL	Host localhost:5433	Database firemex_qa; verify/fix the port mismatch in Section 4.
Home Assistant	http://localhost:8124	Container listens on 8123; config is persisted under homeassistant_config/.
FiremeX API/backend	http://localhost:8081	Start the application separately; API tests may require a valid token.

FiremeX frontend	UI test default http://localhost:5174/FiremeX	Start the application separately. Confirm the application's actual base path/case.
AI inference service	http://localhost:8101	Provide separately if detection tests require inference.
RTSP cameras/NVR		Enabled for computer web camera & CCTV camera
RabbitMQ, Notification gateway, mock fire panel		

3. Network and ports

Service	Host port	Container/service port	Purpose
PostgreSQL	5432	5433	Host-based application/test client connection
Home Assistant	8123	8124	Local Home Assistant UI/API
FiremeX backend	8080	8081	API and registration/login routes
FiremeX frontend	5173	5174	Browser-based application

The database port mapping is host:container. A backend running directly on the workstation should normally connect to localhost:5432. A backend running on the same Compose network should connect to db:5433.
Only the PostgreSQL and Home Assistant ports are declared in docker-compose.qa.yml. The backend/frontend ports must be managed by the separate FiremeX application setup.

4. FiremeX application under test

The application backend and frontend are maintained/deployed separately from this QA repository. Before testing:

1. Start the backend using the application team's supported instructions.
2. Configure it to use the QA PostgreSQL database and the correct connection endpoint for where it runs:
    - Workstation process: localhost:5432 when using the recommended mapping.
    - Compose-network process: db:5433 when using the recommended mapping.
3. Start the frontend using the application team's supported instructions.
4. Configure the frontend API URL or development proxy to target the backend.
5. Open /ping on the backend and verify an HTTP success response before running API tests.

6. Open the frontend login/registration page and verify its API calls in the browser Network panel.

The QA API fixture defaults to http://localhost:8080 and accepts FIREMEX_API_URL or API_BASE_URL overrides. UI tests commonly default to http://localhost:5173/FiremeX and accept BASE_URL in test modules.

5. Test automation setup

5.1 API automation

The API suite is under API-Automation/. It uses pytest and Playwright's API request context.
From the repository root:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r .\API-Automation\requirements.txt
```

Configure the API URL and any credentials required by the selected tests in API-Automation/.env (copy the example as a starting point, then replace placeholders). The fixture reads that file if it exists. Do not commit it.

```
FIREMEX_API_URL=http://localhost:8080
FIREMEX_TEST_EMAIL=<QA account email>
FIREMEX_TEST_PASSWORD=<QA account password>
FIREMEX_API_TOKEN=<scoped QA token if required>
```

Run the API tests:

```
Set-Location .\API-Automation
python -m pytest
```

The valid-login tests require FIREMEX_TEST_EMAIL and FIREMEX_TEST_PASSWORD. Authenticated endpoint tests may require a valid FIREMEX_API_TOKEN/API_TOKEN. Use dedicated QA accounts and data.

5.2 UI automation

UI automation scripts are in automation folders. They use Selenium and a Chromium browser. Install Selenium and a compatible browser/driver in the chosen Python environment before running those tests; the API requirements.txt does not declare Selenium.
Set FIREMEX_BASE_URL to the actual frontend URL where supported, and FIREMEX_HEADLESS=1 for the dashboard suite in headless environments.
Example:

```
$env:FIREMEX_BASE_URL = "http://localhost:5173/FiremeX"
$env:FIREMEX_HEADLESS = "1"
python -m pytest .\tests
```

The UI suite includes test code with fallback credentials. Override them with dedicated QA test users and never use production accounts. A failed authentication setup can make later UI assertions fail before the target feature is reached.

6. Environment validation / tests

Run these checks before a test cycle and record the results:

Check	Validation	Expected result
Compose services	docker compose -f .\docker-compose.qa.yml ps	PostgreSQL and Home Assistant are running.
Database readiness	docker exec firemex_qa_db pg_isready -U admin -d firemex_qa -p 5432	Reports accepting connections on the configured internal port.
Database host port	Test-NetConnection localhost -Port 5433 plus docker port firemex_qa_db	Host port maps to the port PostgreSQL actually listens on.
Home Assistant	Browse to http://localhost:8124	Login/setup page responds.
Backend	Browse/curl the deployed backend /ping route	HTTP success and JSON response.
Frontend	Open the configured frontend URL	Login/registration UI loads.
Frontend-to-backend	Submit a controlled test request and inspect browser Network tab	Request reaches the intended backend; no proxy/CORS/network error.
API automation	Run python -m pytest from API-Automation	Required tests pass with configured QA credentials.
UI automation	Run the selected Selenium suite	Browser can load the app and test account can authenticate.

Camera-online checks, model inference, alert delivery, historical video playback, mock fire-panel calls, and notifications are conditional on separately provisioned services and test data. They cannot be considered passed merely because the two Compose containers are healthy.

7. Data handling and environment refresh

- Use synthetic or approved test data; avoid production personal data and credentials.
- The PostgreSQL volume qa_postgres_data persists across container recreation.
- docker compose down stops and removes the containers but preserves the named database volume.
- docker compose down -v deletes the named database volume and its data. Run it only when a full database reset is explicitly intended.
- Home Assistant state is persisted in the host-mounted homeassistant_config/ directory; removing its container does not reset that directory.
- Back up QA data before refreshes that could alter the schema or test records.
- Record environment defects separately from application defects, using the ENV- prefix where the team’s issue-tracking process supports it.

8. Access, secrets, and network controls

- Restrict QA database and Home Assistant access to authorized testers.
- Keep .env files, API tokens, passwords, and Home Assistant tokens out of source control, test reports, screenshots, and chat logs.
- Use separate, least-privilege QA accounts for administrators, operators, and automated tests.
- Do not expose PostgreSQL, Home Assistant, or the backend directly to the public internet.
- For a shared staging deployment, use HTTPS at the ingress/reverse proxy and restrict camera/RTSP traffic to an approved isolated network.
- Use sandbox notification services and a mock fire-panel endpoint if those integrations are added; the current QA Compose file does not configure them.
- Rotate any development credentials that have been shared or used outside a local isolated setup.